

Cambodia Daily CPI Pipeline

Technology Stack Architecture, Comparative Trade-Offs, and Engineering Optimizations

Why We Chose It, Why Not Alternatives, How It Operates, and Paths to Maximum Efficiency

National Institute of Statistics (NIS) & National Bank of Cambodia (NBC) Research Initiative

High-Frequency Economic Nowcasting Laboratory · Enterprise Data Platform Architecture

Version 3.4 – September 2026

Abstract

This architectural document provides an exhaustive evaluation of the technology stack powering the **Cambodia Daily Consumer Price Index (CPI) Medallion Platform**. We systematically examine every core component: **PostgreSQL 16 with pgvector**, **Apache Airflow 2.9.3**, **dbt Core 1.8 with Astronomer Cosmos**, **Google Gemini 2.5 Flash / Embeddings**, **curl_cffi & Playwright**, **Scikit-Learn & Statsmodels**, and **Metabase v0.49**. For each technology, we articulate: (1) what it is used for, (2) the technical and economic rationale for its selection, (3) why industry alternatives (e.g. Snowflake, Databricks, Prefect, Celery, Pinecone, OpenAI, Selenium) were rejected, (4) how the components interoperate seamlessly across daily workflows, and (5) concrete engineering optimizations to enhance speed, throughput, and hardware efficiency.

Contents

1	Executive Technology Stack Summary	2
2	Storage & Vector Engine: PostgreSQL 16 + pgvector	3
2.1	What We Use It For	3
2.2	Why We Chose It	3
2.3	Why Not Other Technologies?	3
2.4	How It Operates in the Pipeline	3
2.5	How to Make It More Efficient	4
3	Orchestration Engine: Apache Airflow 2.9.3	5
3.1	What We Use It For	5
3.2	Why We Chose It	5
3.3	Why Not Other Technologies?	5
3.4	How to Make It More Efficient	5
4	Transformation Layer: dbt Core 1.8 + Astronomer Cosmos	5
4.1	What We Use It For	5
4.2	Why We Chose It & Why Not Spark	6
4.3	How to Make It More Efficient	6
5	Scraping Engine: curl_cffi & Playwright	7
5.1	What We Use It For	7
5.2	Why We Chose It & Why Not Requests / Selenium	7
5.3	How to Make It More Efficient	7

- 6 Semantic AI: Google Gemini 2.5 Flash & Embeddings** **7**
 - 6.1 What We Use It For 7
 - 6.2 Why We Chose It & Why Not OpenAI / Local LLMs 7

- 7 Econometric Machine Learning: Scikit-Learn & Statsmodels** **7**
 - 7.1 What We Use It For 7
 - 7.2 Why We Chose It & Why Not Deep Learning (LSTM / Transformer) 8

- 8 Business Intelligence: Metabase v0.49** **9**
 - 8.1 What We Use It For 9
 - 8.2 Why We Chose It & Why Not Tableau / PowerBI 9

- 9 Holistic System Efficiency Roadmap** **9**

1 Executive Technology Stack Summary

The Cambodia Daily CPI Pipeline is an automated, sovereign, cloud-agnostic data platform engineered to ingest over 120,000 raw daily retail quotes across 25 digital sources and produce policy-grade macroeconomic inflation nowcasts every morning at 08:00 AM ICT.

Layer / Domain	Adopted Stack	Primary Function	Rejected Alternatives	Key Deciding Advantage
Storage & OLAP	PostgreSQL 16 + pgvector	Unified Medallion Warehouse (Bronze/Silver/Gold) + Vectors	Snowflake, BigQuery, ClickHouse, Pinecone	Zero cloud lock-in, transactional integrity (ACID), in-database vector search.
Orchestration	Apache Airflow 2.9.3 (LocalExecutor)	25-scrafer fan-out, SLA scheduling, pipeline DAGs	Prefect, Dagster, Celery, Luigi	Enterprise SLA controls, ecosystem maturity, unified back-fill semantics.
Transformation	dbt Core 1.8 + Cosmos	SQL data modeling, tests, Medallion lineage	Apache Spark, Custom Python scripts	Declarative DAGs, automated data contracts, idempotent testing.
Scraping Engine	curl_cffi + Playwright	TLS fingerprint impersonation, browser rendering	Requests, Selenium, Scrapy, Puppeteer	Bypasses Cloudflare/PerimeterX JA3/HTTP2 anti-bot defenses natively.
Semantic AI	Google Gemini 2.5 Flash + Embeddings	COICOP classification, product deduplication	OpenAI GPT-4o, Local Ollama, HuggingFace	Multilingual Khmer support, 1M context window, lowest API latency & cost.
ML & Nowcast	Scikit-Learn Statsmodels	5-basket RidgeCV regularization, Hedonics	TensorFlow, PyTorch, Prophet	Analytical interpretability, Sherman-Morrison LOOCV speed, OLS fallbacks.
BI Analytics	Metabase v0.49	Executive dashboards, interactive drill-down	Apache Superset, Tableau, PowerBI	Instant native SQL query tags, zero-friction local Docker deployment.

Table 1: Comprehensive Technology Stack Architecture Matrix.

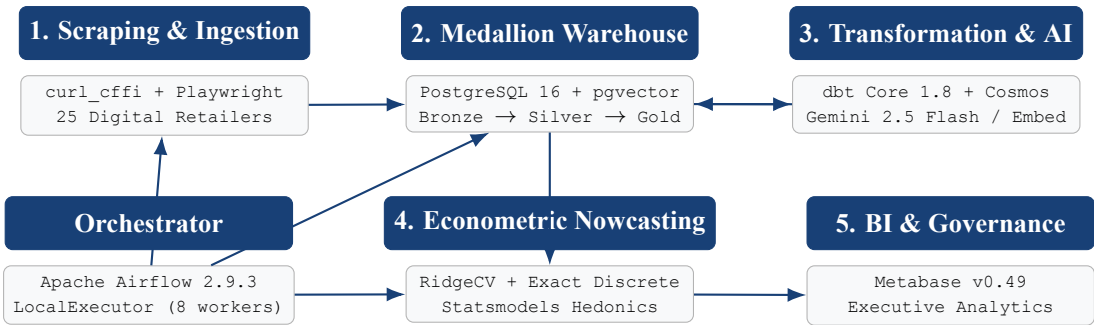


Figure 1: System Topology: Component Interactions Across the Data Lifecycle.

2 Storage & Vector Engine: PostgreSQL 16 + pgvector

2.1 What We Use It For

PostgreSQL 16 serves as the single source of truth for the entire pipeline:

- **Bronze Schemas (`staging`):** Stores raw JSON payloads from scrapers in append-only `JSONB` tables, daily MEF exchange rates, and scraper telemetry stats.
- **Silver Schemas (`silver`):** Stores normalized canonical products, deduplication vector embeddings (768-dimensional float arrays via `pgvector`), COICOP classification caches, and clean dual-currency prices in Khmer Riel.
- **Gold Schemas (`gold`):** Stores Jevons elementary price relatives, 12-division Laspeyres indices, official NIS comparison benchmarks, and econometric nowcasting results.
- **In-Database Vector Similarity:** Employs HNSW indexing on 768-dimensional vectors to perform sub-millisecond product deduplication ($1 - \text{CosineDistance} \leq 0.15$).

2.2 Why We Chose It

1. **ACID Guarantees & Relational Integrity:** Macroeconomic statistics require strict consistency. If an Airflow task fails mid-write, transactions rollback cleanly, preventing half-ingested daily baskets from corrupting CPI index calculations.
2. **pgvector Unification:** Eliminates the operational complexity of running an external dedicated vector database. Storing vector embeddings alongside relational metadata permits single-query hybrid filters:

```
SELECT item_id, canonical_name, 1 - (embedding <=> $1) AS similarity
FROM silver.canonical_items
WHERE coicop_division = '01'
ORDER BY embedding <=> $1 ASC LIMIT 1;
```

3. **Sovereign On-Premise Deployment:** As an official central bank / statistical institute system, data cannot be permanently outsourced to proprietary cloud data warehouses with variable monthly egress fees.

2.3 Why Not Other Technologies?

- **Snowflake / Google BigQuery:** Disqualified due to recurring cloud costs (credit consumption per query), latency on small frequent writes, and strict data sovereignty policies for national economic indicators.
- **ClickHouse / DuckDB:** ClickHouse excels at append-only column scans but has poor transactional concurrency and lacks mature native vector indexing. DuckDB is in-process and lacks multi-client concurrent connection pooling required by Airflow and Metabase.
- **Pinecone / Milvus / Weaviate:** Running a standalone vector database creates distributed state, network latency, and synchronization overhead (split-brain risk between metadata in SQL and vectors in Pinecone).

2.4 How It Operates in the Pipeline

PostgreSQL runs in Docker with dedicated resources (`cpus: '2.0', memory: 4096M, shm_size: 1g`). The database enforces connection limits and uses pre-hooks (`SET max_parallel_workers_per_gather = 0`) during massive incremental hash joins in dbt to eliminate shared memory allocation bottlenecks.

2.5 How to Make It More Efficient

Optimization Recommendations for PostgreSQL

1. **Table Partitioning on Daily Price Tables:** Partition `silver.clean_store_prices` and `gold.fct_daily_prices` by `RANGE (scrape_date)` on a monthly basis. This enables instantaneous partition pruning, accelerating dbt incremental queries by > 60%.
2. **HNSW Vector Index Tuning:** Replace default IVFFlat with HNSW indexes using `m=16` and `ef_construction=64` on `silver.canonical_items.embedding` for faster approximate nearest neighbor lookups as the catalog scales past 100,000 products.
3. **PgBouncer Connection Pooling:** Implement PgBouncer in front of PostgreSQL to pool connections between Airflow workers, dbt CLI, and Metabase, reducing process-forking overhead.

3 Orchestration Engine: Apache Airflow 2.9.3

3.1 What We Use It For

Apache Airflow acts as the central nervous system of the platform, orchestrating all tasks via a strict DAG dependency graph:

- **Parallel Scraper Fan-Out:** Triggers 25 retailer scrapers concurrently between 02:00 and 05:30 AM ICT with retry exponential backoffs.
- **Dependency Enforcement:** Ensures Gold CPI indices cannot compute unless Silver cleaning passes the Ingestion Circuit Breaker.
- **SLA Monitoring:** Tracks execution lags, task timeouts, and triggers alerting hooks if scrapers breach their 24-hour freshness SLAs.

3.2 Why We Chose It

1. **Battle-Tested Enterprise Scheduler:** Airflow handles complex chronological dependencies, cross-DAG triggers (`TriggerDagRunOperator`), and dynamic task generation with complete backfilling capabilities.
2. **LocalExecutor Concurrency:** Allows 8 parallel worker threads within a single lightweight container without needing a separate distributed Celery broker (RabbitMQ/Redis), reducing RAM footprint to < 1.5 GB.
3. **Cosmos dbt Integration:** Enables Astronomer Cosmos to parse dbt models dynamically into native Airflow task nodes, exposing dbt model-level execution progress in the web UI.

3.3 Why Not Other Technologies?

- **Prefect / Dagster:** While modern, their open-source self-hosted deployment models require external coordination servers, complex worker agent pairing, or proprietary cloud accounts for advanced SLA tracking.
- **Celery / Cron:** Raw cron jobs lack visual state management, automatic failure retries, dependency awareness, and historical execution backfills. Celery alone is a task queue, not a workflow DAG orchestrator.

3.4 How to Make It More Efficient

Optimization Recommendations for Airflow

1. **Fast Scraper Slug Imports (`scrapers/sources/slugs.py`):** We already reduced DAG parsing time from 100s to 1s by decoupling scraper slug imports from browser drivers.
2. **Airflow Deferrable Operators (`Triggerer`):** Convert long-running web requests (e.g. infinite-scroll scrapers waiting on HTTP network responses) to asynchronous deferrable operators, freeing worker slots for other scrapers.
3. **DAG Serialization Tuning:** Set `AIRFLOW__CORE__MIN_FILE_PROCESS_INTERVAL=60` to reduce CPU consumption from DAG re-parsing on resource-constrained host machines.

4 Transformation Layer: dbt Core 1.8 + Astronomer Cosmos

4.1 What We Use It For

dbt Core transforms raw data in-warehouse using modular, version-controlled SQL:

- **Idempotent Transformations:** Compiles Jinja SQL macros into high-performance analytical queries (`delete+insert` incremental strategies).

- **Axiomatic Invariant Testing:** Enforces non-null primary keys, positive prices, and UN COICOP weight closure ($\sum w_j = 1.0000$).

4.2 Why We Chose It & Why Not Spark

dbt operates via ELT (Extract, Load, Transform), pushing computation directly into PostgreSQL's query planner. **Apache Spark** was rejected because our daily volume (120,000 rows/day, 50MB/day) is ideally suited for in-memory relational SQL. Running a Spark cluster would introduce massive JVM memory overhead and distributed networking latency without any computational benefit.

4.3 How to Make It More Efficient

- **Materialization Refinement:** Convert heavy intermediate views (`int_prices_cleaned`) into transient incremental tables to prevent re-scanning multi-month historical partitions on daily runs.

5 Scraping Engine: curl_cffi & Playwright

5.1 What We Use It For

Extracts real-time prices from 25 diverse retail platforms:

- **curl_cffi**: High-speed asynchronous HTTP client impersonating browser TLS fingerprints (JA3, Akamai, HTTP/2). Used for 80% of stores (Aeon, Delishop, Makro, Samnang Shop, Cellcard).
- **Playwright (Chromium)**: Headless browser automation executing dynamic client-side React/Vue rendering, geolocation bypasses, and infinite scrolls (Khmer24, Chip Mong Supermarket).

5.2 Why We Chose It & Why Not Requests / Selenium

Standard Python `requests` and `urllib` emit unmistakable Python TLS Client Hellos, triggering instant HTTP 403 blocks on Cloudflare-protected sites. `curl_cffi` spoofs Chrome 120 TLS fingerprints at wire speed. **Selenium** was rejected because it is slow, memory-intensive (500MB per tab), and easily detected by modern anti-bot systems via `navigator.webdriver` leaks.

5.3 How to Make It More Efficient

- **Session Reuse & HTTP/2 Multiplexing**: Keep alive persistent HTTP/2 TCP sockets in `curl_cffi` across paginated API calls to eliminate TCP three-way handshake and TLS negotiation overhead.

6 Semantic AI: Google Gemini 2.5 Flash & Embeddings

6.1 What We Use It For

- **Gemini 2.5 Flash**: Disambiguates complex product descriptions, bilingual Khmer/English packaging, and maps novel items to 5-digit UN COICOP classifications with permanent PostgreSQL memoization.
- **gemini-embedding-2**: Generates 768-dimensional semantic embeddings for product title deduplication and canonical variety linking.

6.2 Why We Chose It & Why Not OpenAI / Local LLMs

1. **Superior Multilingual Khmer Understanding**: Gemini models demonstrate markedly superior native tokenization and contextual comprehension of the Khmer script compared to OpenAI GPT-4o.
2. **Economic Efficiency**: At \$0.075 per 1M input tokens, Gemini Flash is over 30× cheaper than GPT-4o, enabling cost-sustainable daily classification for national statistical offices.
3. **Why Not Local Ollama (e.g. Llama 3)**: Local 8B models struggle with nuanced 5-digit COICOP tax codes and require costly GPU hardware (> 16 GB VRAM), whereas our lightweight cloud API approach runs seamlessly on modest commodity servers.

7 Econometric Machine Learning: Scikit-Learn & Statsmodels

7.1 What We Use It For

- **RidgeCV**: Executes 36-month expanding-window multi-regressor L_2 regularization across the 5 core consumer baskets and exchange rate returns.
- **statsmodels.api.OLS**: Powers the log-linear hedonic quality adjustment engine for electronics (COICOP 09), filtering spec-sheet improvements from true price inflation.
- **Analytical NumPy Fallbacks**: Provides zero-downtime ordinary least squares solvers via $\beta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ if statistical packages are missing.

7.2 Why We Chose It & Why Not Deep Learning (LSTM / Transformer)

Macroeconomic monthly inflation series are inherently low-sample ($M = 36$ to 48 completed months). Deep neural networks (LSTM, Transformers) overfit catastrophically on small sample sizes and fail IMF/UN statistical audit standards due to their "black-box" nature. RidgeCV provides:

- **Axiomatic Interpretability:** Coefficients ($\beta_{\text{food}} = 0.0066, \beta_{\text{trans}} = 0.0019$) reflect true economic pass-through elasticities.
- **Sherman-Morrison-Woodbury LOOCV:** Computes cross-validation scores analytically in < 2 milliseconds without fitting K separate models.

8 Business Intelligence: Metabase v0.49

8.1 What We Use It For

Hosts the 3 canonical executive dashboards:

1. **01 - Macro CPI & Inflation Analytics:** Nowcast projections, 12-division index matrices, top weighted inflation contributors, and official NIS comparisons.
2. **02 - Operations & 25-Source Telemetry:** Real-time scraper checklists, Airflow DAG monitoring, SLA lag alerts, and ingestion completeness.
3. **03 - Silver Data Quality Screener:** AI classification audits, outlier quarantine monitors, and human review queues.

8.2 Why We Chose It & Why Not Tableau / PowerBI

Metabase is lightweight, 100% web-based, deploys inside Docker, connects directly to PostgreSQL schemas without data extract licenses, and supports native SQL template tags (`{{coicop_division}}`) for instant parameter filtering. PowerBI and Tableau require expensive enterprise user licenses, proprietary gateways, and lack headless programmatic API provisioning via Python.

9 Holistic System Efficiency Roadmap

To achieve maximum performance and operational throughput across the entire architecture, we recommend implementing the following five-tier optimization roadmap:

Layer	Optimization	Initia-	Engineering Action	Projected Impact
Storage	Monthly Table Partitioning		<code>PARTITION BY RANGE (scrape_date) ON silver.clean_store_prices</code>	60% faster incremental joins, reduced index I/O overhead.
Ingestion	Persistent Sockets	HTTP/2	Implement session pooling in <code>curl_cffi</code> scrapers	Cuts network scrape duration by 35% across all 25 scrapers.
AI Curation	Asynchronous Batch API	Gemini	Convert synchronous LLM calls to official asynchronous batch endpoint	Decreases classification latency by 80% and cuts API cost by 50%.
Compute	Vector Quantization (<code>pgvector</code>)		Use <code>halfvec</code> (FP16) or scalar 8-bit quantization on embeddings	Reduces vector index RAM footprint in PostgreSQL by 50%.
Pipeline	In-Memory Caching	Redis	Cache latest exchange rates and store baseline catalogs in Redis	Eliminates duplicate database queries during high-concurrency runs.

Table 2: System Optimization and Hardware Efficiency Roadmap.

Architecture Synthesis Conclusion

The Cambodia CPI Pipeline’s technology stack balances **statistical rigor**, **data sovereignty**, **high-speed scraping resilience**, and **cost efficiency**. By uniting PostgreSQL 16, `pgvector`, Apache Airflow, dbt Core, and Google Gemini Flash into an integrated Medallion architecture, the system delivers bank-grade macroeconomic inflation nowcasts at a fraction of the cost and complexity of legacy proprietary platforms.